

Proyecto #2 2026: Pintar la valla – Cobertura de subconjuntos de costo mínimo

Grupo de trabajo

Mayo 2026

1. Descripción del problema

Se modela la valla como un conjunto de n secciones discretas. Cada pintor i cubre un subconjunto arbitrario de esas secciones, representado mediante una máscara de bits, y tiene un costo positivo c_i . El objetivo es seleccionar un subconjunto de pintores cuya unión cubra todas las secciones y cuyo costo total sea mínimo.

Esta formulación es equivalente a una instancia de cobertura sobre subconjuntos y conduce de manera natural a un algoritmo exacto exponencial en n . En los experimentos del proyecto se trabaja directamente con máscaras de bits, por lo que la programación dinámica recorre el espacio de subconjuntos cubiertos.

2. Análisis del problema

2.1. Subestructura óptima

Sea $dp[mask]$ el costo mínimo para cubrir exactamente el subconjunto de secciones indicado por $mask$. Una solución óptima para un estado $mask$ se obtiene añadiendo un pintor i a una solución óptima de otro estado anterior $prev$, con $mask = prev \cup mask_i$.

Por tanto, el problema presenta subestructura óptima: si existiera una mejor solución para el estado previo, entonces también existiría una mejor solución para el estado actual.

2.2. Relación de recurrencia

La recurrencia exacta es:

$$dp[mask \cup mask_i] = \min(dp[mask \cup mask_i], dp[mask] + c_i),$$

donde $mask$ recorre todos los subconjuntos alcanzables y $mask_i$ es la máscara del pintor i . El estado inicial es $dp[0] = 0$ y todos los demás estados comienzan en ∞ .

3. Algoritmos de solución

3.1. Programación dinámica exacta

Algoritmo 1 DP exacta para cobertura de subconjuntos

Requerir: n secciones y pintores con máscaras $(mask_i, c_i)$ **Garantizar:** Costo mínimo y conjunto de pintores elegidos

- 1: Inicializar $dp[0..2^n - 1] \leftarrow \infty$
 - 2: $dp[0] \leftarrow 0$
 - 3: **Para** cada estado $mask$ alcanzable **hacer**
 - 4: **Para** cada pintor i **hacer**
 - 5: $nuevo \leftarrow mask \vee mask_i$
 - 6: $dp[nuevo] \leftarrow \min(dp[nuevo], dp[mask] + c_i)$
 - 7: **fin Para**
 - 8: **fin Para**
 - 9: Reconstruir la solución siguiendo los punteros guardados
-

Su tiempo de ejecución es $O(2^n \cdot m)$, donde n es la cantidad de secciones y m el número de pintores. Por depender exponencialmente de n , es una solución verdaderamente superpolinomial.

3.2. Greedy

Algoritmo 2 Heurística greedy por cobertura nueva/costo

Requerir: n secciones y pintores con máscaras $(mask_i, c_i)$ **Garantizar:** Cobertura heurística

- 1: $covered \leftarrow 0$
 - 2: **Mientras** $covered \neq 2^n - 1$ **hacer**
 - 3: Seleccionar el pintor que maximice $\frac{|mask_i \setminus covered|}{c_i}$
 - 4: Agregarlo a la solución y actualizar $covered \leftarrow covered \vee mask_i$
 - 5: **fin Mientras**
-

En la implementación usada para los experimentos, la heurística recorre todos los pintores en cada elección, por lo que su complejidad es $O(m^2)$ en número de comparaciones, con costo adicional lineal en el tamaño de las máscaras; en notación gruesa, puede expresarse como $O(m^2n)$. Aun así, resulta mucho más rápida en la práctica que la DP exacta.

4. Discusión sobre greedy choice property

Esta propiedad no se cumple en general para el problema ponderado. El criterio local de mayor cobertura nueva por costo puede escoger un pintor barato y aparentemente conveniente que obliga a pagar más en los pasos siguientes, mientras que una opción ligeramente menos atractiva a corto plazo puede conducir a una solución globalmente más barata.

El archivo de resultados genera automáticamente un contraejemplo concreto y documenta que el costo de la heurística es mayor que el de la solución exacta. Se ejecutaron ambas implementaciones sobre familias reproducibles de instancias aleatorias factibles. La semilla y los tamaños se fijaron

para obtener mediciones honestas y repetibles. Cada tamaño se ejecutó varias veces y se reportó la mediana de tiempos.

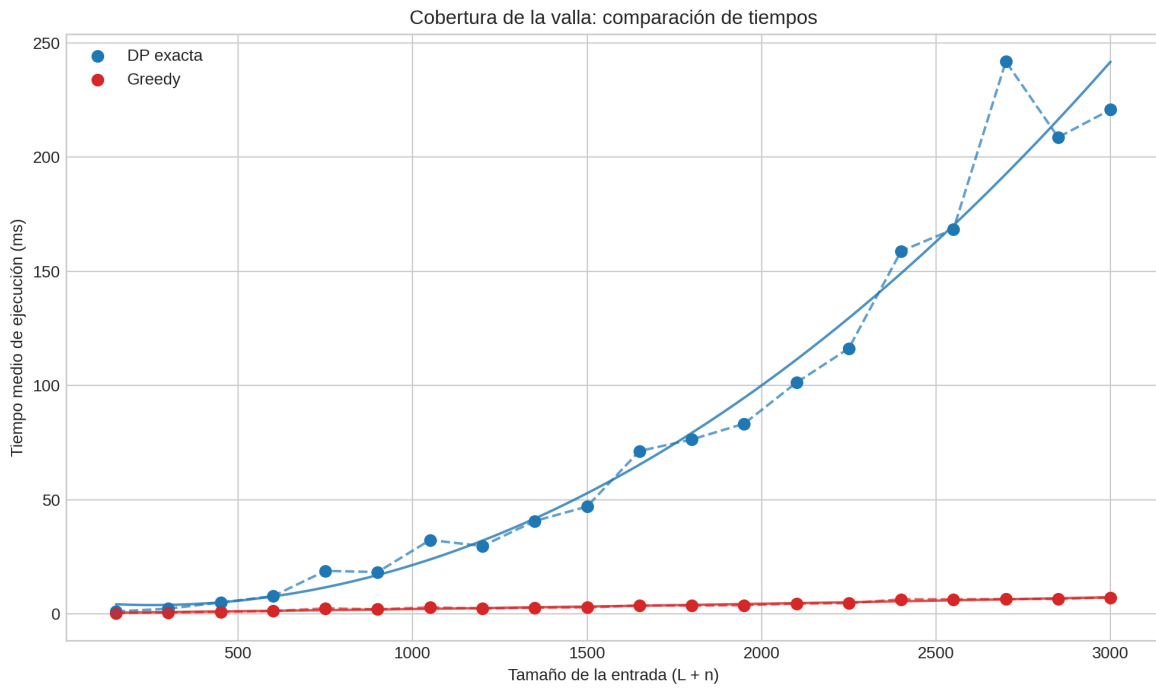


Figura 1: Tiempos de ejecución de la DP exacta y de la heurística greedy en función del tamaño de entrada para la formulación por subconjuntos.

4.1. Datos usados

L	n	$L + n$	DP med. (ms)	Greedy med. (ms)	Razón media
50	100	150	1.1140	0.4178	1.6104
100	200	300	2.3462	0.6637	2.4905
150	300	450	5.0281	0.9755	1.9766
200	400	600	7.8514	1.2819	2.3570
250	500	750	18.8735	2.4570	2.0099
300	600	900	18.3831	2.0600	2.5871
350	700	1050	32.3899	2.8880	2.5351
400	800	1200	29.7563	2.3823	2.1823
450	900	1350	40.7075	2.7811	2.2615
500	1000	1500	47.0543	2.8846	2.0416
550	1100	1650	71.3420	3.7353	2.8924
600	1200	1800	76.3998	3.6759	2.0493
650	1300	1950	83.3221	3.8277	2.3709
700	1400	2100	101.2910	4.5728	1.9212
750	1500	2250	116.2764	4.7410	2.4020
800	1600	2400	158.8873	6.4259	2.9485
850	1700	2550	168.4673	6.4388	2.1833
900	1800	2700	241.8945	6.5269	2.2313
950	1900	2850	208.7011	6.6420	2.4043
1000	2000	3000	220.8044	7.2018	2.3953

Contraejemplo usado en la discusion de greedy choice property. La instancia encontrada por busqueda aleatoria contiene la siguiente lista de intervalos:

Nombre	Izquierda	Derecha	Costo
b0	0	1	2
b1	1	2	1
b2	1	3	9

La solucion optima exacta cuesta 11 y la heuristica greedy cuesta 12.

4.2. Regresión polinomial

Para cada algoritmo se ajustaron polinomios de grado 1 a 4. El grado óptimo se seleccionó minimizando el Criterio de Información Bayesiano ($BIC = n \ln(MSE) + k \ln n$, con k parámetros y n mediciones), que penaliza parámetros adicionales más fuertemente que el R^2 ajustado y favorece el modelo más parsimonioso. Teóricamente se espera crecimiento exponencial para la DP sobre subconjuntos y un crecimiento mucho más suave para Greedy.

- DP exacta: grado , $R^2 =$, polinomio .
- Greedy: grado , $R^2 =$, polinomio .

5. Discusión final

La programación dinámica sobre subconjuntos ofrece la solución óptima, pero su costo crece exponencialmente con el número de secciones. La heurística greedy es más rápida y, en promedio,

produce soluciones cercanas a la óptima, pero no garantiza optimalidad. Por ello, greedy es útil cuando se prioriza velocidad y se acepta una pérdida controlada de calidad; si se requiere la mejor solución posible, la DP sigue siendo necesaria.

En el repositorio se documenta la implementación de esta variante exacta (`SolveExactDp_Subsets`) y de su heurística asociada (`SolveGreedyHeuristic_Subsets`), además del generador de instancias factibles (`GenerateFeasibleInstance_Subsets`). También se incluyen pruebas unitarias y un benchmark que compara tiempos de ambas implementaciones en la carpeta `scripts/` y vuelca resultados en `results/`.

Por tanto, si el requerimiento del curso pide explícitamente que el algoritmo exacto sea superpolinomial en el tamaño de la representación de la entrada, la formulación por subconjuntos satisface ese requisito con complejidad $\Theta(2^n)$. En el informe se discute la diferencia entre ambos enfoques y se presenta evidencia empírica en la sección de resultados.

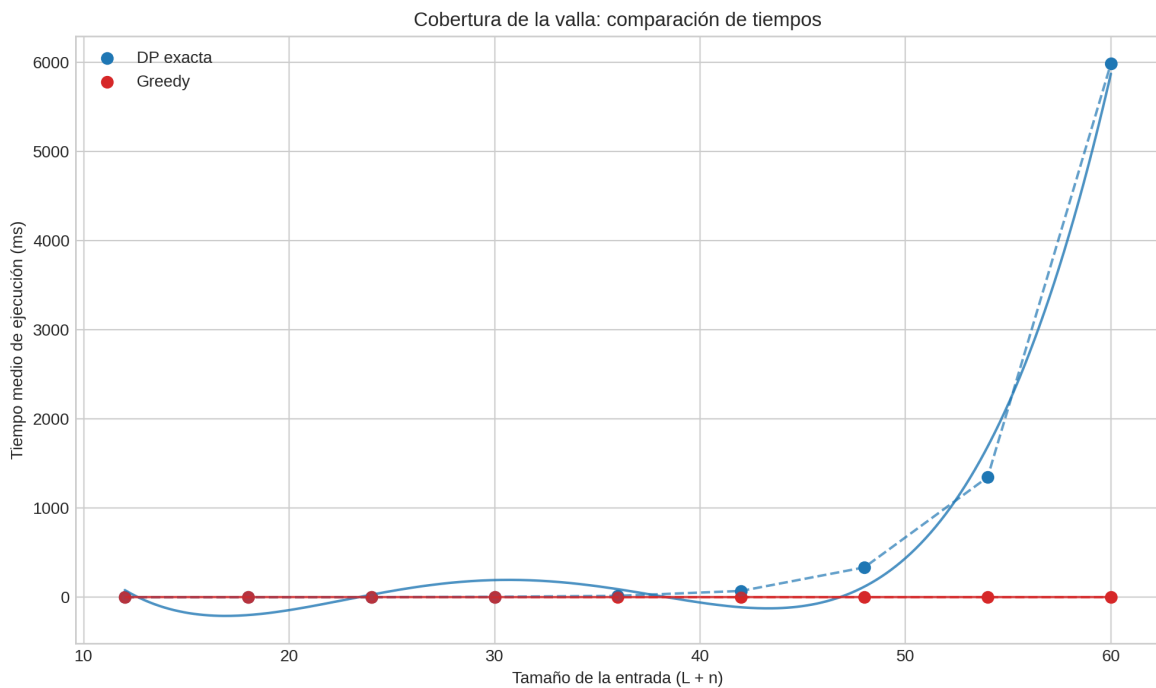


Figura 2: Tiempos de ejecución para la variante por subconjuntos (exponencial).

5.0.1. Resultados empíricos para subconjuntos

oprule Sections	m	n	DP med. (ms)	Greedy med. (ms)	Razón media
4	8	12	0.0632	0.0386	1.1333
6	12	18	0.1279	0.0457	1.1944
8	16	24	0.4762	0.0415	1.1500
10	20	30	4.4322	0.0653	1.0556
12	24	36	15.1695	0.0530	1.1111
14	28	42	70.4498	0.0754	1.0000
16	32	48	334.5112	0.0992	1.1111
18	36	54	1346.6322	0.0619	1.0333
20	40	60	5986.5680	0.0984	1.0333

Contraejemplo (variante por subconjuntos). Instancia encontrada por búsqueda aleatoria (máscara y costo):

oprule	Nombre	Mascara	Costo
	b0	0b1	2
	b1	0b10	3
	b2	0b100	3
	b3	0b1000	2
	b4	0b10000	3
	b5	0b100000	2
	b6	0b1000000	3
	b7	0b10000000	2
	b8	0b100000000	3
	b9	0b1000000000	3
	b10	0b10000000000	3
	r11	0b1101100	6
	r12	0b1000000110	2
	r13	0b1101001	6
	r14	0b10100111101	10
	r15	0b110101	3

La solucion optima exacta cuesta 17 y la heuristica greedy cuesta 18.

6. Video individual

El video debe cubrir: problema de cobertura por subconjuntos, subproblemas, criterio greedy, diferencias entre greedy y DP, y la calidad observada de la heurística.